January 2026 CSE 216
Online on PL/SQL
Subsection: B1+B2
Time: 1 h 15 mins + 5 minutes submission
Marks: 20

---

1. A department runs a tenure-bonus round with a fixed cash budget. Each employee's bonus percentage depends on both their length of service and whether they sit below the midpoint of their job's salary range (`(JOBS.MIN_SALARY + JOBS.MAX_SALARY) / 2`):

Condition (checked top to bottom)	Bonus % of salary
salary below job midpoint and tenure ≥ 5 years	15 %
tenure ≥ 10 years	12 %
tenure ≥ 5 years	8 %
otherwise	4 %

Employees are served most senior first (`HIRE_DATE`, then `EMPLOYEE_ID`). A running total of bonuses paid so far is kept. The first employee whose bonus would push the running total past the budget gets nothing, and so does everyone after them (seniority order) — even if a later, cheaper employee would still have fit. Awards that *are* granted are written to a table `BONUS_AWARD`

Task

(a) Write a function

```
GET_TENURE_YEARS ( P_EMP_ID IN NUMBER ) RETURN NUMBER
```

returning the employee's completed service in years, rounded to 2 decimals (`ROUND(MONTHS_BETWEEN(SYSDATE, HIRE_DATE) / 12, 2)`). If the id does not exist, return -1 (handle it in the exception section).

(b) Write a procedure

```
AWARD_TENURE_BONUS ( P_DEPT_ID       IN  NUMBER,
                     P_BUDGET        IN  NUMBER,
                     P_AWARDED_COUNT OUT NUMBER,
                     P_TOTAL_BONUS   OUT NUMBER,
                     P_FIRST_SKIPPED OUT NUMBER )
```

that:

- first deletes any previous `BONUS_AWARD` rows for this department's employees (so the procedure can be re-run);
- walks the department's employees most-senior-first, computes the bonus % from the table above (calling `GET_TENURE_YEARS` for the tenure), and `bonus = ROUND(salary * pct / 100, 2)`;
- inserts (`employee_id, tenure_years, bonus_amount, USER, SYSDATE`) into `BONUS_AWARD` for every granted award, keeping a running total;
- stops granting from the first employee that would breach `P_BUDGET` onward;
- returns: `P_AWARDED_COUNT` = how many awards were granted, `P_TOTAL_BONUS` = money actually committed, `P_FIRST_SKIPPED` = `EMPLOYEE_ID` of the first employee denied by the budget, or `NULL` if everyone fitted;
- if the department has no employees, prints `No employees in department <id>` and returns `0, 0, NULL`.

(c) Write an anonymous block that runs it for department `80` with a budget of `40000` and prints the three `OUT` values.

2. Whenever an employee is transferred from one department to another, the employee should normally report to the manager of the new department.

The `DEPARTMENTS` table stores the manager of each department in its `MANAGER_ID` column.
Create the following table:

```
CREATE TABLE DEPARTMENT_TRANSFER_LOG (
    EMPLOYEE_ID        NUMBER,
    OLD_DEPARTMENT_ID  NUMBER,
    NEW_DEPARTMENT_ID  NUMBER,
    OLD_MANAGER_ID     NUMBER,
    NEW_MANAGER_ID     NUMBER,
    CHANGED_BY         VARCHAR2(30),
    CHANGED_ON         DATE
);
```

Task:

Create a row-level trigger on `EMPLOYEES` that fires whenever `DEPARTMENT_ID` is updated.

If the department actually changes:

- Look up the `MANAGER_ID` of the employee's new department from the `DEPARTMENTS` table.
- If the new department has a manager, automatically change the employee's `MANAGER_ID` to that manager.
- If the new department has no manager (`MANAGER_ID` is `NULL`), leave the employee's existing manager unchanged.
- Insert one row into `DEPARTMENT_TRANSFER_LOG` containing:
    - employee ID
    - old department ID
    - new department ID
    - old manager ID
    - final new manager ID
    - database user
    - current date/time
- If an `UPDATE` assigns the employee to the same department they already belong to, do not write a log row.

For this question, you may assume that all department IDs used in the test cases are valid and non-`NULL`.

Demonstrate your trigger using suitable `UPDATE` statements and show:

```
SELECT *
FROM DEPARTMENT_TRANSFER_LOG;
```

Also display the affected employee rows afterward to verify that their `MANAGER_ID` values were changed correctly.

Test driver:

In the standard HR schema:

- Department `50` has manager `121`.
- Department `120` has no manager.
- Employees `104`, `105`, and `106` initially belong to department `60`.

Test 1: Move to a department that has a manager

```
UPDATE EMPLOYEES

SET DEPARTMENT_ID = 50
```

```
WHERE EMPLOYEE_ID = 104;
```

Employee 104 should now have:

```
DEPARTMENT_ID = 50

MANAGER_ID    = 121
```

because employee 121 manages department 50.

Test 2: Move to a department with no manager

```
UPDATE EMPLOYEES

SET DEPARTMENT_ID = 120

WHERE EMPLOYEE_ID = 105;
```

Department 120 has no manager.

Therefore employee 105's existing manager should remain unchanged.

Test 3: Assign the same department

```
UPDATE EMPLOYEES

SET DEPARTMENT_ID = 60

WHERE EMPLOYEE_ID = 106;
```

Since employee 106 already belongs to department 60:

- the manager is not modified;
- no row is inserted into DEPARTMENT_TRANSFER_LOG.

Verify the log

```
SELECT EMPLOYEE_ID,

       OLD_DEPARTMENT_ID,

       NEW_DEPARTMENT_ID,

       OLD_MANAGER_ID,

       NEW_MANAGER_ID

FROM DEPARTMENT_TRANSFER_LOG
```

```
ORDER BY CHANGED_ON;
```

The important rows should be equivalent to:

EMPLOYEE_ID	OLD_DEPT	NEW_DEPT	OLD_MANAGER	NEW_MANAGER
104	60	50	103	121
105	60	120	103	103

There should be no log row for employee `106`, because the value of `DEPARTMENT_ID` did not actually change.

Finally:

```
ROLLBACK;
```